

Setup and Operations Guide

Everything needed to install, run, deploy, and maintain the project — written for whoever is setting this up, whether or not they built it. The companion **Software Requirements** PDF lists what to install before you start here.

Contents

1. Installing each piece of software
2. Complete project setup from a fresh computer
3. Running the frontend, backend, and database
4. Environment variables, configuration, and deployment
5. Common errors and their solutions
6. Troubleshooting guide
7. Folder structure explanation
8. Building the production version
9. Updating and maintaining the project

1. Installing each piece of software

Node.js (and npm)

Windows / macOS: download the LTS installer from nodejs.org and run it. npm is included automatically.

macOS (Homebrew):

```
brew install node@22
```

Ubuntu / Debian Linux:

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -  
sudo apt-get install -y nodejs
```

Verify:

```
node --version    # v22.x.x or later  
npm --version     # 10.x.x or later
```

Git

Windows: download from git-scm.com/download/win and run the installer.

macOS: `brew install git`, or install Xcode Command Line Tools (`xcode-select --install`).

Ubuntu / Debian: `sudo apt-get install git`

Verify: `git --version`

Docker + Docker Compose

Windows / macOS: install Docker Desktop from docker.com/products/docker-desktop — this includes Compose automatically.

Ubuntu / Debian Linux:

```
curl -fsSL https://get.docker.com | sudo sh  
sudo usermod -aG docker $USER    # log out and back in after this
```

Verify:

```
docker --version  
docker compose version
```

2. Complete project setup from a fresh computer

These steps take a brand-new computer to a running local copy of the app.

Step 1 — Install prerequisites

Node.js and Git at minimum (see Section 1). Install Docker too if you'll also test the production build.

Step 2 — Get the code onto the machine

```
git clone <your-repository-url>
cd <project-folder>
```

Step 3 — Set up the backend

```
cd backend
cp .env.example .env
```

Open `.env` and set at minimum `JWT_SECRET` (generate one with `openssl rand -base64 48`) and `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD` — the first login you'll use to sign in.

```
npm install
npm run db:migrate # creates the SQLite database and tables
npm run db:seed    # creates the first admin account
```

Step 4 — Set up the frontend

```
cd ../frontend
npm install
```

No `.env` file is needed in local development — the dev server forwards API requests to the backend automatically.

Step 5 — Run both and sign in

See Section 3. Once both are running, open `http://localhost:5173` and sign in with the admin credentials from Step 3.

That's the entire setup. There is no database server to install, no message queue, no cache — just two `npm install` commands and two migration commands.

3. Running the frontend, backend, and database

Database

There is nothing to run separately — SQLite lives in one file (`backend/data/app.db` by default) that the backend opens directly. It is created automatically by `npm run db:migrate`.

Command	What it does
<code>npm run db:migrate</code>	Creates/updates tables to match the current schema. Safe to re-run.
<code>npm run db:seed</code>	Creates the first admin account (skips if one already exists).
<code>npm run db:generate</code>	Development only — after changing <code>schema.ts</code> , generates a new migration file to commit.
<code>npm run db:backup</code>	Writes a point-in-time backup to <code>backend/data/backups/</code> .

Backend (API server)

```
cd backend
npm run dev
```

Starts the API on **`http://localhost:4000`** with auto-restart on file changes.

Frontend (web UI)

```
cd frontend
npm run dev
```

Starts the UI on **`http://localhost:5173`** with hot-reload. This is the URL to open in your browser.

Run both at once: two terminals, one for each command above, left running while you work.

4. Environment variables, configuration, and deployment

Backend environment variables (backend/.env)

Variable	Purpose	Example
NODE_ENV	development or production	development
PORT	Port the API listens on	4000
DATABASE_PATH	Path to the SQLite file	./data/app.db
JWT_SECRET	Signs login sessions — must be secret/random	openssl rand -base64 48
JWT_EXPIRES_IN	How long a login lasts	8h
CORS_ORIGIN	Where the frontend is served from	http://localhost:5173
SEED_ADMIN_*	First admin account, used once by db:seed	—

Root environment variables (.env, Docker Compose only)

Variable	Purpose
DOMAIN	The domain Caddy will get an HTTPS certificate for
JWT_SECRET, JWT_EXPIRES_IN	Same as above, passed into the container
CORS_ORIGIN	Should match https://
SEED_ADMIN_*	First admin account for the production database

Never commit a real .env file to Git — .gitignore already excludes it. The .env.example files are safe templates only.

Deploying to production

The app deploys as one Docker Compose stack: one container running the Node app (API + built frontend, one port), and one container running Caddy as a reverse proxy that gets and renews HTTPS certificates automatically. A small VPS (1 vCPU / 1GB RAM) comfortably serves 10–20 users.

```
# 1. Install Docker and Git on the server
# 2. Get the code
git clone <your-repository-url>
cd <project-folder>

# 3. Point your domain's DNS A record at the server's IP first —
#     Caddy needs this to succeed before it can get a certificate.

# 4. Configure
cp .env.example .env
nano .env      # set DOMAIN, JWT_SECRET, CORS_ORIGIN, SEED_ADMIN_*

# 5. Build and start
docker compose up -d --build

# 6. Watch logs the first time
docker compose logs -f
```

Visit <https://<your-domain>> and sign in with your SEED_ADMIN_* credentials. **Change that password immediately** from Settings › My Account.

Updating a production deployment

```
git pull
docker compose up -d --build
```

Rebuilds the image and applies any new migrations automatically on startup — no manual database steps.

Setting up nightly backups

```
crontab -e
```

Add:

```
0 2 * * * /full/path/to/project/scripts/backup.sh >> /full/path/to/project/backups/backup.log 2>&1
```

Runs nightly at 2 AM, writing a timestamped backup to `./backups/` on the host and deleting anything older than 14 days.

5. Common errors and their solutions

Error	Cause	Solution
Missing required environment variable: JWT_SECRET	No .env file, or not filled in	cp .env.example .env in backend/, then set a real JWT_SECRET
EADDRINUSE :::4000	Port already in use by another process	Stop the other process, or change PORT in backend/.env
Frontend loads but every request fails	Backend isn't running, or CORS_ORIGIN mismatch	Start the backend; make sure CORS_ORIGIN matches the browser URL
403 "Request rejected for security reasons"	Missing X-Requested-With header (CSRF safeguard)	The frontend sends this automatically; add it manually when testing the API by hand
Signed out immediately after signing in	Secure cookie blocked over plain http:// in production	Access production over https:// (Caddy provides this); http://localhost is fine in dev
npm install fails on better-sqlite3	Missing build tools on that Linux machine	sudo apt-get install -y python3 make g++, then retry (handled automatically in Docker)
Caddy never gets a certificate	DNS doesn't point at the server, or ports 80/443 blocked	Confirm the domain's A record and firewall/cloud provider allow inbound 80 and 443
Admin password forgotten	—	See "Resetting a password by hand" below

Resetting a password by hand

Run from backend/ (adjust the value):

```
node -e "
  console.log(require('bcryptjs').hashSync('temporary-new-password', 12));
"
```

Then update that user's password_hash column to the printed value using a SQLite tool pointed at backend/data/app.db (or /app/data/app.db inside the running container).

6. Troubleshooting guide

Work through these in order — most problems are caught by the first two or three steps.

1. **Check both processes are running.** Development needs the backend and frontend dev servers running at the same time, in two terminals.
2. **Check the backend's terminal output.** Every request is logged; an error there is almost always more informative than what the browser shows.
3. **Check the browser console and Network tab (F12).** Console shows JS errors; Network shows the real HTTP status and response body.
4. **Confirm .env is actually filled in,** not just copied from the example.
5. **In Docker, check container logs:** `docker compose logs -f app / caddy`.
6. **Confirm the database file exists** and has the right permissions.
7. **When in doubt, restart clean** — stop and restart the dev servers, or `docker compose down && docker compose up -d --build`. This resolves most "it just stopped working" situations without losing data.
8. **Still stuck?** Re-read the exact error message against the Common Errors table in Section 5.

7. Folder structure explanation

```
project/
+-- backend/                                Node.js/TypeScript API
|   +-- src/
|       +-- app.ts                          Express app: middleware, routes, static serving
|       +-- server.ts                       Starts the HTTP server
|       +-- config.ts                       Reads/validates environment variables
|       +-- db/                             schema.ts, client.ts, migrate.ts, seed.ts, backup.ts
|       +-- middleware/                     auth.ts, errorHandler.ts
|       +-- validators/                     schemas.ts (Zod – every input rule)
|       +-- lib/                             workStatus.ts, audit.ts, jwt.ts, errors.ts
|       \-- routes/                         auth, tasks, users, audit – one file per resource
|   +-- scripts/                           migrate-excel.ts – one-time legacy import tool
|   +-- tests/                             backend test suite (vitest)
|   +-- drizzle/                           generated SQL migrations – commit these
|   \-- data/                             SQLite database file (gitignored)
|
+-- frontend/                              React/TypeScript UI
|   \-- src/
|       +-- main.tsx                       Entry point: providers
|       +-- App.tsx                         Route definitions
|       +-- pages/                          Login, Home, CategoryPage, Settings
|       +-- components/                     layout/, dashboard/, table/, modals/, common/
|       +-- hooks/                          React Query hooks for tasks and users
|       +-- context/                        Auth state
|       \-- lib/                            API client, work-status helpers, utilities
|
+-- scripts/backup.sh                       Host-level nightly backup wrapper
+-- docs/                                  This documentation
+-- Dockerfile                             Multi-stage build -> one production image
+-- docker-compose.yml                     The two-container production deployment
\-- Caddyfile                              Reverse proxy + automatic HTTPS
```

Why frontend and backend are separate folders: they are independent Node packages with their own package.json and dependencies. This keeps browser-only code and server-only code from ever accidentally depending on each other, and lets each be built, tested, and deployed on its own.

8. Building the production version

You will not normally need to do this by hand — `docker compose up -d --build` does it automatically. To build and run each piece manually (useful for debugging a build issue):

```
# Frontend: produces frontend/dist (static HTML/CSS/JS)
cd frontend
npm run build

# Backend: produces backend/dist (compiled JavaScript)
cd ../backend
npm run build

# Wire them together the same way the Dockerfile does, then run:
mkdir -p public
cp -r ../frontend/dist/* public/
NODE_ENV=production node dist/server.js
```

The backend serves the built frontend directly — in production there is only one server and one port, no separate frontend server and no CORS to configure.

9. Updating and maintaining the project

Changing the database schema

- Edit backend/src/db/schema.ts.
- Run `npm run db:generate` from backend/ — writes a new SQL file into backend/drizzle/.
- Commit both the schema change and the new migration file to Git.
- Deploy as usual — the new migration is applied automatically on container startup.

Never edit an already-committed migration file. Write a new one instead, to keep every environment's history consistent.

Adding a new API endpoint

Add a route in backend/src/routes/, validate its input in validators/schemas.ts, and call it from a new hook in frontend/src/hooks/. Every existing resource follows the same shape (route → Zod validation → Drizzle query → response) — use one as a template.

Keeping dependencies up to date

```
cd backend && npm outdated # then npm update, or bump versions individually
cd frontend && npm outdated
```

Run the test suites after updating (npm test in both folders) before deploying.

Rotating the JWT secret

Changing JWT_SECRET immediately signs every user out (their tokens no longer verify). Safe to do at any time — update .env and redeploy.

Restoring from a backup

Stop the stack, replace the database file inside the app_data volume with the chosen file from ./backups/, and start the stack again. Always test a restore occasionally — a backup you've never restored from is not a verified backup.

Getting help

Every error the backend produces is logged with full detail — start there. The codebase is intentionally small and consistently structured specifically so a future developer, even one unfamiliar with the project, can find the relevant file quickly.